Vehicle Control Project: Modeling and PID Control

24-677 Special Topics: Linear Control Systems

Prof. M. Bedillion
Based on materials originally developed by Prof. Ding Zhao.

Due: October 30, 2026, 11:59 pm.

- As with regular homework, we will use Gradescope to grade.
- Submit **your_controller.m** to Gradescope under **Programming-P1** and your solutions in PDF format to **Project-P1**. Insert `controller_results.png` in the PDF. We will test **your_controller.m** and manually check the written answers.
- This repository targets **Webots R2025a**. Install that release from the official Webots releases page; using an older release may rewrite the world in an incompatible format.
- The controller backend is entirely MATLAB; Python, NumPy, SciPy, and Matplotlib are not required. The project has been verified with 64-bit **MATLAB R2025a**. On Windows, also install the MATLAB Support for MinGW-w64 C/C++ Compiler add-on, which Webots needs to load its controller and vehicle-driver libraries. NOTE: We have had mixed results with Matlab 2025a. If you are having "Load Library" issues, we recommend installing 2025a for a seamless experience.
- If Webots does not detect MATLAB, set the MATLAB command under **Tools** → **Preferences** → **General**. On Windows, select `MATLAB.exe` under `bin\win64` (for example, `C:\Program Files\MATLAB\R2025a\bin\win64\MATLAB.exe`), not the launcher under `bin`.
- Please familiarize yourself with Webots documentation, specifically their User Guide and their Webots for Automobiles section, if you encounter difficulties in setup or use. It will help to have a good understanding of the underlying tools that will be used in this assignment. To that end, completing at least Tutorial 1 in the user guide is highly recommended.
- Open `worlds/automotive_new.wbt` from Webots and press Run. Do not start `main_matlab.m` directly: Webots must establish the controller connection and environment first.

1 Introduction

This project uses Webots, an open-source robotics simulator, and MATLAB to study control methods for autonomous vehicles. The supplied repository contains a Webots R2025a world, a native MATLAB controller template, the reference trajectory, and the course grader. No Python runtime or Python plotting package is required. Detailed installation and launch instructions are provided on the title page and in `README.md`.

For all supported operating systems:

1. Install 64-bit Webots R2025a and a compatible 64-bit MATLAB release. The Windows configuration has been runtime-tested with MATLAB R2025a; macOS and Ubuntu support is implemented but should be validated on the target machine.

2. Install and configure a MATLAB-supported C compiler by running `mex -setup C` in MATLAB:

   - Windows: install MATLAB's MinGW-w64 compiler support add-on.
   - macOS: install the Xcode Command Line Tools and select a MATLAB-supported Clang compiler.
   - Ubuntu: install a GCC version supported by the installed MATLAB release.

3. Install the native Webots package for the operating system:

   - Windows: use the R2025a `.exe` installer.
   - macOS: use the R2025a `.dmg` package.
   - Ubuntu: use the R2025a `.deb` package or tarball. Do not use the Snap package because its sandbox does not support MATLAB controllers.

4. Configure the MATLAB executable in Webots if it is not detected automatically.

5. Open `worlds/automotive_new.wbt`, confirm that the Tractor robot uses the `main_matlab` controller, and press Run.

Buggy is a time-honored CMU tradition (you can learn more about it here and here). In this project, you will create a controller that drives the simulated tractor around the Buggy course.

We will study this problem from various angles as we move through the rest of the course, applying newly learned control techniques on the autonomous vehicle system. We will follow the steps listed below to learn more about the system and synthesize several different kinds of controllers:

1. Examine the provided nonlinear control model

2. Linearize the state space system equations [Project 1]

3. Develop a PID controller for the system [Project 1]
4. Check the controllability and stabilizability of the system [Project 2]
5. Design a full-state feedback controller using pole placement [Project 2]
6. Design an optimal controller [Project 3]
7. Implement an extended Kalman filter (EKF) for simultaneous localization and mapping (SLAM) [Project 4]

The project has been divided into 4 parts to reflect this:

Project 1
(a) Linearize the state space model
(b) Design a PID lateral and PID longitudinal controller

Project 2
(a) Check the controllability and stabilizability of the linearized system
(b) Design a full-state feedback controller

Project 3
(a) Design an optimal controller

Project 4
(a) Implement EKF SLAM to control the vehicle without perfect sensor input
(b) Race with other Buggy competitors in the class

2 Model

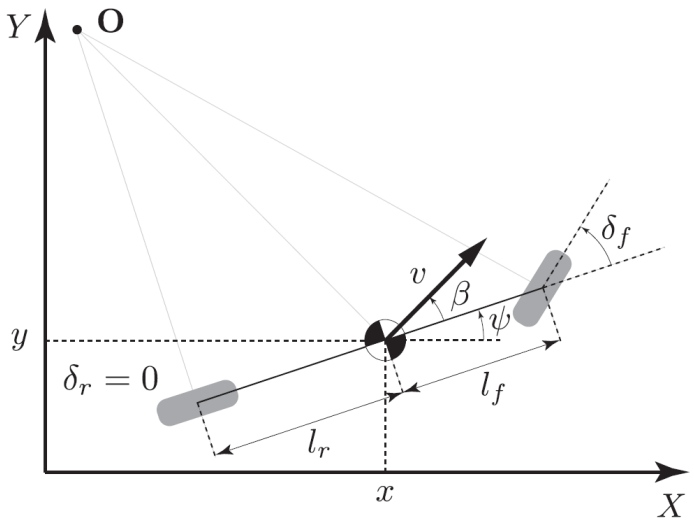


Figure 1: Bicycle model [2]

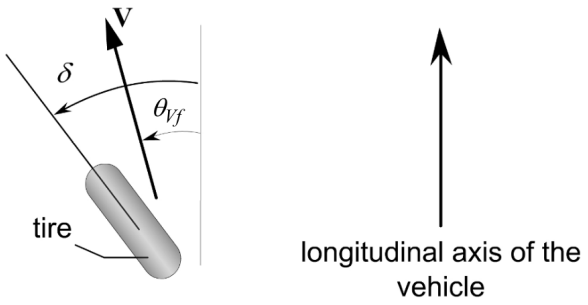


Figure 2: Tire slip angle [2]

We will make use of a bicycle model for the vehicle, which is a popular model in the study of vehicle dynamics. Shown in Figure 1, the car is modeled as a two-wheel vehicle with two degrees of freedom, described separately in longitudinal and lateral dynamics. The model parameters are defined in Table 2.

2.1 Lateral dynamics

Ignoring road bank angle and applying Newton's second law of motion along the y-axis:

$$ma_y = F_{yf} \cos \delta_f + F_{yr}$$

where $a_y = \left(\frac{d^2 y}{dt^2} \right)_{inertial}$ is the inertial acceleration of the vehicle at the center of geometry in the direction of the y axis, F_{yf} and F_{yr} are the lateral tire forces of the front and rear

wheels, respectively, and δ_f is the front wheel angle, which will be denoted as δ later. Two terms contribute to a_y: the acceleration $\ddot{y}$, which is due to motion along the y-axis, and the centripetal acceleration. Hence:

$$a_y = \ddot{y} + \dot{\psi}\dot{x}$$

Combining the two equations, the equation for the lateral translational motion of the vehicle is obtained as:

$$\ddot{y} = -\dot{\psi}\dot{x} + \frac{1}{m}(F_{yf}\cos\delta + F_{yr})$$

Moment balance about the axis yields the equation for the yaw dynamics as

$$\ddot{\psi} I_z = l_f F_{yf} - l_r F_{yr}$$

The next step is to model the lateral tire forces F_{yf} and F_{yr}. Experimental results show that the lateral tire force is proportional to the "slip angle" for small slip angles when the vehicle speed is large enough—that is, when $\dot{x} \geq 0.5$ m/s. A tire's slip angle is the angle between the tire orientation and the vehicle velocity vector. The slip angles of the front and rear wheels are

$$\alpha_f = \delta - \theta_{Vf}$$
$$\alpha_r = -\theta_{Vr}$$

where θ_{Vp} is the angle between the velocity vector and the longitudinal axis of the vehicle, for $p \in \{f, r\}$. A linear approximation of the tire forces are given by

$$F_{yf} = 2C_\alpha \left(\delta - \frac{\dot{y} + l_f\dot{\psi}}{\dot{x}}\right)$$
$$F_{yr} = 2C_\alpha \left(-\frac{\dot{y} - l_r\dot{\psi}}{\dot{x}}\right)$$

where C_α is called the cornering stiffness of the tires. If $\dot{x} < 0.5$ m/s, we just set F_{yf} and F_{yr} both to zeros.

2.2 Longitudinal dynamics

Similarly, a force balance along the vehicle longitudinal axis yields:

$$\ddot{x} = \dot{\psi}\dot{y} + a_x$$
$$ma_x = F - F_f$$
$$F_f = fmg$$

where F is the total tire force along the x-axis, and F_f is the force due to rolling resistance at the tires, and f is the friction coefficient.

2.3 Global coordinates

In the global frame we have:

$$\dot{X} = \dot{x}\cos\psi - \dot{y}\sin\psi$$
$$\dot{Y} = \dot{x}\sin\psi + \dot{y}\cos\psi$$

2.4 System equations

Gathering all of the equations, if $\dot{x} \geq 0.5$ m/s, we have:

$$\ddot{y} = -\dot{\psi}\dot{x} + \frac{2C_\alpha}{m}(\cos\delta\left(\delta - \frac{\dot{y} + l_f\dot{\psi}}{\dot{x}}\right) - \frac{\dot{y} - l_r\dot{\psi}}{\dot{x}}) \tag{1}$$

$$\ddot{x} = \dot{\psi}\dot{y} + \frac{1}{m}(F - fmg) \tag{2}$$

$$\ddot{\psi} = \frac{2l_fC_\alpha}{I_z}\left(\delta - \frac{\dot{y} + l_f\dot{\psi}}{\dot{x}}\right) - \frac{2l_rC_\alpha}{I_z}\left(-\frac{\dot{y} - l_r\dot{\psi}}{\dot{x}}\right) \tag{3}$$

$$\dot{X} = \dot{x}\cos\psi - \dot{y}\sin\psi \tag{4}$$

$$\dot{Y} = \dot{x}\sin\psi + \dot{y}\cos\psi \tag{5}$$

Otherwise, since the lateral tire forces are zeros, we only consider the longitudinal model.

2.5 Measurements

The observable states are:

$$y = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\psi} \\ X \\ Y \\ \psi \end{bmatrix}$$

2.6 Physical constraints

The system satisfies the constraints that:

$$|\delta| \leqslant \tfrac{\pi}{6}\ rad$$
$$F \geqslant 0 \text{ and } F \leqslant 10000\ N$$
$$\dot{x} \geqslant 10^{-5}\ m/s$$

Table 1: Model parameters.

Name	Description	Unit	Value
$(\dot{x}, \dot{y})$	Vehicle's velocity along the direction of vehicle frame	m/s	State
(X, Y)	Vehicle's coordinates in the world frame	m	State
ψ, $\dot{\psi}$	Body yaw angle, angular speed	rad, rad/s	State
δ or δ_f	Front wheel angle	rad	Input
F	Total input force	N	Input
m	Vehicle mass	kg	1000
l_r	Length from rear tire to the center of mass	m	0.82
l_f	Length from front tire to the center of mass	m	1.18
C_α	Cornering stiffness of each tire	N	20000
I_z	Yaw inertia	kg m^2	3004.5
F_{pq}	Tire force, $p \in \{x, y\}$,$q \in \{f, r\}$	N	Depends on input force
f	Rolling resistance coefficient	N/A	0.028
`delT`	Simulation timestep	sec	0.032

3 Resources

3.1 Simulation

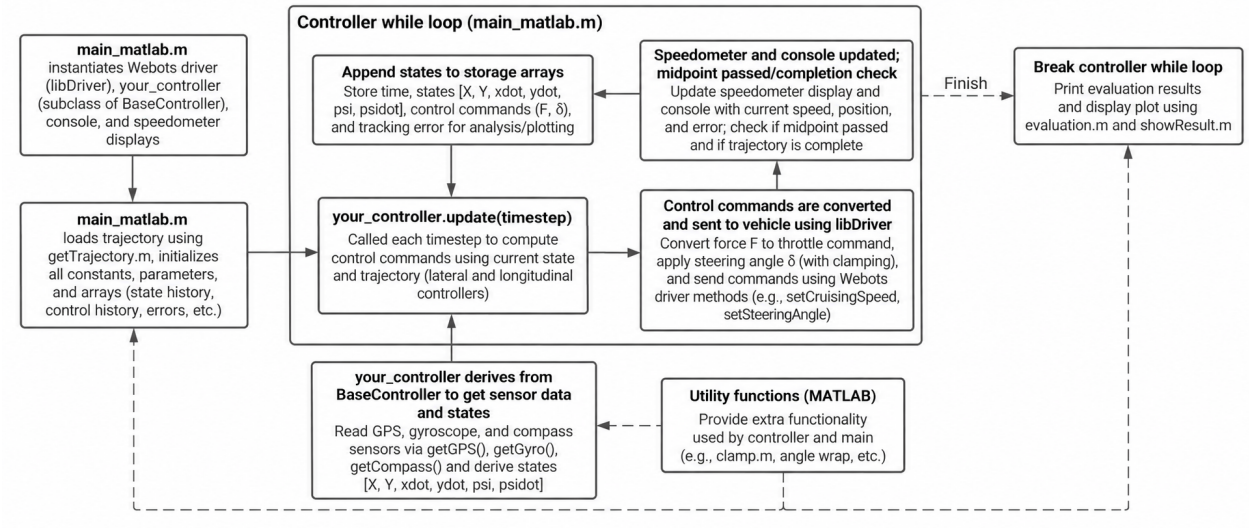


Figure 3: MATLAB simulation code flow

The MATLAB controller files are in `controllers/main_matlab`. Webots launches `main_matlab.m`, which initializes the vehicle-driver library, loads `buggyTrace.csv`, starts the sensors, and runs the simulation loop every 32 ms. On each iteration it calls the `update` method in `your_controller.m` to obtain the desired steering angle δ and longitudinal force F. The main controller clamps δ to $\pm\pi/6$, converts F to a throttle command with saturation, updates the in-world displays, and records the state history.

Design the controller only in `your_controller.m`, specifically in its two labeled lateral and longitudinal sections. You may add properties and initialize them in the class constructor when values must persist between calls. Do not rename the controller directory or `main_matlab.m`; Webots identifies the controller using those names.

3.2 BaseController Background

The `your_controller` MATLAB class derives from `BaseController`. The vehicle is equipped with simulated GPS, gyroscope, and compass devices. `BaseController.m` enables these devices and derives the measured vehicle states as summarized below.

Table 2: State Derivation.

Name	Explanation
(X, Y)	From GPS readings
$(\dot{x}, \dot{y})$	From the derivative of GPS readings
ψ	From the compass readings
$\dot{\psi}$	From the gyroscope readings

3.3 Trajectory Data

The trajectory is given in `buggyTrace.csv`. It contains the coordinates of the trajectory as (x, y). The satellite map of the track is shown in Figure 4.

When the tractor reaches the destination, `evaluation.m` prints the component scores and total grade. `showResult.m` then creates a native MATLAB figure, saves it as `controllers/main_matlab/controller_results.png`, and keeps the window open until it is closed.

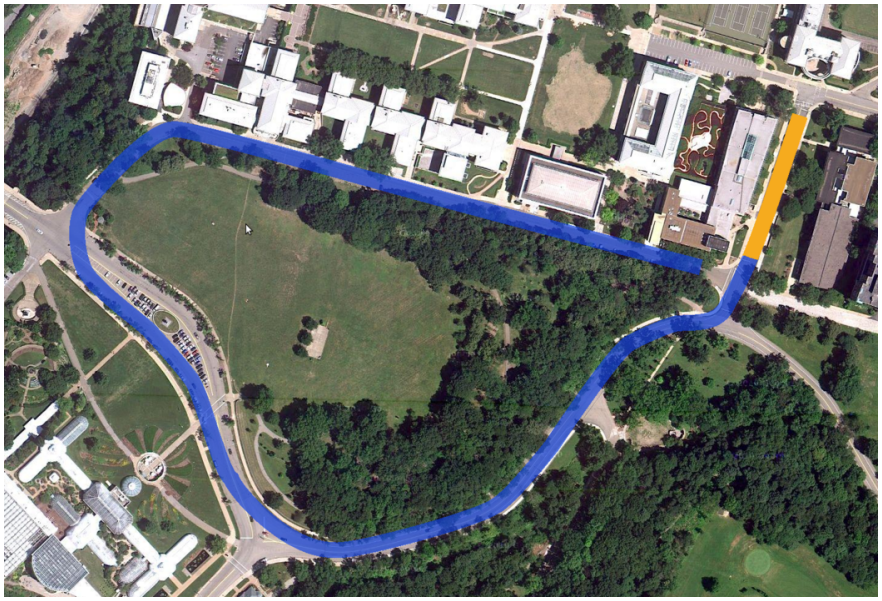


Figure 4: Buggy track [3]

4 Problems

Exercise 1. Model Linearization. As mentioned in class, model linearization is usually the first step in control design for non-linear systems. During this assignment, you will approximate the given model with a linear model. For this we will use the following state vector

$$\chi = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \psi \\ \dot{\psi} \\ X \\ Y \end{bmatrix}.$$

For the below assume that for values of $\dot{x} < 0.5$ m/s that the system is identical, i.e. there is no need to linearize two separate systems.

- Using the given nonlinear state equations, derive the linearized state equations for the arbitrary operating point χ_0.
- Use the parameters in Table 2 and the prior results to find the linearized model about operating point $\dot{y} = \dot{\psi} = \psi = 0$, $\dot{x} = 6$. At this operating point the car is driving straight at the track center, and something interesting happens to the dynamics. NOTE: If you are having extra constant terms in your state equation, be sure you are properly using and applying the equilibrium inputs.
- Convert the model you previously found to a transfer function matrix assuming that the system outputs are ψ and $\dot{x}$, the heading angle and longitudinal speed. You should

find that you have two decoupled transfer functions, i.e. at this point the steering angle affects only ψ and the force affects only the longitudinal velocity $\dot{x}$. What are the poles and zeros of the two transfer functions?

Exercise 2. Controller Synthesis in Simulation. The driver functions that control the car take the desired steering angle δ and a throttle input - ranging from 0 to 1 - which is derived from the desired longitudinal force F.

For this question, you have to design a PID longitudinal controller and a PID lateral controller for the vehicle. A PID is an error-based controller that requires tuning proportional, integral, and derivative gains. As a PID allows us to minimize the error between a set point and process variable without deeper knowledge of the system model, we will not need our result from Exercise 1 (though it will be useful in future project parts, and you could certainly design by hand at least for the simple longitudinal dynamics model).

Design the two controllers in `controllers/main_matlab/your_controller.m`. Implement the labeled sections of the `update` method and assign the MATLAB outputs `delta` and `F`.

Check the performance of your controller by running the Webots simulation. You can use real-time, fast, or no-rendering mode. If you complete the track, MATLAB prints the grading results, opens a native MATLAB performance figure, and saves the figure as `controller_results.png`. The plot shows the reference and driven trajectories together with the steering, force, velocity, heading, yaw-rate, and cross-track-error histories.

Submit `your_controller.m` and the saved completion plot as described on the title page. The repository grader assigns 30 points each for loop completion, average distance, and maximum-distance compliance, plus up to 10 points for completion time. Its principal thresholds are:

1. Baseline completion time: 400 s.
2. Full average-distance score when the mean cross-track error is at most 3.0 m; the score decreases to zero at 6.0 m.
3. Maximum-distance compliance threshold: 6.0 m, scored as the fraction of recorded samples within this distance.
4. Loop-completion threshold: each retained reference waypoint must lie within 12.0 m of the driven path for the full completion score.

Some hints that may be useful:

- A PID controller needs to remember certain values between consecutive calls to `update()`, such as accumulated errors for the integral term. In MATLAB, these values can be stored as class properties, which retain their values between method calls. For example:

```
classdef your_controller < BaseController
    properties
        accumulated_error = 0;
    end
```

```
    methods
        function obj = your_controller(trajectory)
            obj@BaseController(trajectory);
        end

        function [delta, F] = update(obj, timestep)
            % Example error value
            current_error = 2;

            % Update stored error
            obj.accumulated_error = ...
                obj.accumulated_error + current_error * timestep;

            % Your control logic goes here
        end
    end
end
```

Unlike regular variables defined inside a function, class properties retain their values between calls to `update()`. For example, `obj.accumulated_error` stores the running sum of errors from previous timesteps, allowing the controller to use past information when calculating the next control input. Without storing this value, the accumulated error would reset each time `update()` is called, preventing the integral component of the PID controller from working correctly. The same approach can be used to store the previous error for the derivative term.

- You will need to use the sensor measurements to compute the error in heading angle and error in longitudinal velocity. While the longitudinal velocity error is straightforward to compute based on your measured states, you will have to think about how to compute the heading error given your current heading ψ and the relationship between your global position and the given trajectory points.

- If you are tuning your lateral controller for a point on the trajectory that is the closest to the vehicle, the controller may struggle on sudden turns. Think about how to add some mechanism that "looks ahead" of your current position before calculating a control command.

- The supplied MATLAB helpers may be useful. For example, `closestNode.m` returns the absolute cross-track error and the index of the closest trajectory waypoint; `wrapToPi.m` wraps heading errors to $[-\pi, \pi)$; and `clamp.m` applies saturation limits.

- If your car does not perform well, it may just be that it requires tuning. You may already have some experience tuning PID controllers, but if not, viewing online resources is recommended. See this Wiki link [4] for more background in the process of manual tuning.

5 Reference

1. Rajamani Rajesh. Vehicle Dynamics and Control. Springer Science & Business Media, 2011.

2. Kong Jason, et al. "Kinematic and dynamic vehicle models for autonomous driving control design." Intelligent Vehicles Symposium, 2015.

3. cmubuggy.org, https://cmubuggy.org/reference/File:Course_hill1.png

4. "PID Controller—Manual Tuning." *Wikipedia*. https://en.wikipedia.org/wiki/PID_controller#Manual_tuning

5. Cyberbotics. "Webots R2025a Release." https://github.com/cyberbotics/webots/releases/tag/R2025a

6. Cyberbotics. "Using MATLAB with Webots." https://cyberbotics.com/doc/guide/using-matlab

7. MathWorks. "MATLAB Support for MinGW-w64 C/C++ Compiler." https://www.mathworks.com/matlabcentral/fileexchange/52848-matlab-support-for-mingw-w64-c-c-com